A GUIDE FOR BUILDING A TEST PROJECT

What a Frege testing API should contain

A good testing API is small enough to explain in a minute and varied enough that every part of Frege has something real to act on. This guide lists the kinds of endpoint to include, what each is for, and which Frege feature it exercises. The theme (a shop, a clinic, a library) does not matter; the coverage does.

1. Authorization: the part that matters most

Frege's hardest job is signing someone in to another company's API and keeping them signed in. Give it every common way in, then give it the failures.

Endpoint kind	What it looks like	What it tests in Frege
Password login	POST with email and password; returns an access token, a refresh token and the user	A sign-in flow with a form step and a call step; mapping token fields to outputs
Sign-up	POST that creates an account and signs it in	Sign-in operations: set aside from the tools and used only by the flow
One-time code	Two calls: ask for a code by email or phone, then verify it	A flow with two forms and two calls; a code the tester can read somewhere
Signed request	A call that needs a timestamp header and an HMAC signature	The signature and current-time values; secret variables that stay hidden
OAuth 2.0 with PKCE	An authorize page and a token endpoint	The redirect step; a client secret kept as a secret
Token refresh	POST a refresh token, get a NEW access token and a NEW refresh token; the old one dies	The refresh part of a flow; renewing quietly before expiry
Short token life	Access token lifetime that can be set to about a minute	Seeing renewal happen in minutes instead of hours
Reuse detection	Replaying a spent refresh token ends the whole sign-in	What Frege does when renewal is refused; the credential-expired hook
Logout and revoke	End one session, or all of them; ended tokens stop working at once	Credential expiry; the next tool call after a sign-out
API key variant	A fixed key in a header or query parameter	Sending a key on every call instead of running a flow

Endpoint kind	What it looks like	What it tests in Frege
Session cookie variant	Login sets a cookie instead of returning a token	Flows that output a session token or read a cookie
Who am I	GET /me returns the signed-in user and their role	The first tool to try after any sign-in
Honest failures	Different 401 reasons (missing, invalid, expired, ended), 403 disabled, 429 locked out with Retry-After	Failure reasons in the audit log; messages the AI can act on

2. Roles: who may do what

Include endpoints at four levels, so the same sign-in can be tested as four different people. The role should be checked on every request, not read once from the token.

Level	Example endpoints	What it tests in Frege
Public	Browse the catalog, read the docs, health check	Tools that need no sign-in
Signed-in user	Own profile, own orders, own addresses	The normal customer path
Staff	Read everyone's orders, act on someone else's behalf	An agent limited to a short list of tools; acting as a customer to see what they see
Admin	List users, change a role, disable an account	Consequential tools; members and roles inside Frege versus roles inside the API

Three behaviours are worth building in: someone else's record answers 404 rather than 403, a 403 names the role that was needed, and a role change takes effect on the very next request.

3. Reading data

Most AI conversations are reads, so give them variety.

Endpoint kind	What it looks like	What it tests in Frege
Paged list	page and page_size parameters, with a total in the answer	Paging is detected from the parameter names; the AI can ask for the next page
Cursor list	A cursor or limit parameter and a next marker	The other common paging style
Filters and sorting	Free text, category, price range, a sort choice	The AI choosing arguments from a plain request
Get one by id	A single record, with a 404 for a missing one	Tools with path parameters

Endpoint kind	What it looks like	What it tests in Frege
Empty result	A list that legitimately returns nothing	How the AI explains “nothing found”
Several languages	A Locale header that changes names and messages	Replies in the user's language
Not JSON	An invoice as an HTML page, or a CSV export	What the AI sees when the answer is not JSON
A very large answer	A list of a thousand rows	Size limits and what gets cut

4. Changing data

Writes are where Frege's safety features earn their keep, so include every kind.

Endpoint kind	What it looks like	What it tests in Frege
Create	POST answering 201 with the new record	A tool the AI must ask permission to use
Replace and update	PUT for the whole record, PATCH for a few fields	Two similar tools the AI must tell apart
Delete	DELETE answering 204 with no body	How an empty answer is shown
Safe retry	POST that accepts an Idempotency-Key and returns the first result on a repeat	Retrying without creating duplicates
An action	POST /orders/{id}/cancel: a verb, not a record	Tools that are not plain create, read, update, delete
Strict validation	422 listing every bad field; 400 for a field the API does not know	The AI reading the error and fixing its own request
State conflict	409 when the action does not fit the current state (paying twice)	The AI explaining why it cannot proceed

In Frege this is where the write setting (deny, ask or allow) is tested, and where evals can check that the AI does not write without being asked.

5. Dangerous actions

Add a few actions whose mistakes are expensive, and put a rule in the docs about when a person must decide.

Endpoint kind	Example	What it tests in Frege
Moves money	Pay an order; approve a refund	Approval before the call; a guardrail that insists on a clear yes

Endpoint kind	Example	What it tests in Frege
Cannot be undone	Delete an account	Warnings in the tool description; confirmation in the agent's instructions
Changes who can do what	Change a role, disable a user	Admin-only tools; the audit log
A threshold rule	Refunds up to a small amount are automatic, larger ones wait for staff	Skills and instructions that carry a business rule, and evals that check it is followed

6. Slow work

Real systems have jobs that do not finish in one request.

Endpoint kind	What it looks like	What it tests in Frege
Start a job	POST answering 202 with a job id and a Location header	An AI that must not report a result too early
Check a job	GET returning running, then succeeded, with progress	Polling, and a helper agent that waits and reports back
A job that fails	Same flow, ending in failed with a reason	Reporting a failure honestly
A deliberate delay	An endpoint that waits a chosen number of seconds	Timeouts and how they are reported

7. Sensitive data

If nothing in the API is sensitive, redaction and output guardrails have nothing to protect. Use made-up values, never real ones.

Endpoint kind	What it looks like	What it tests in Frege
A staff-only field	A customer record with a taxpayer number that customers never see	Redaction rules that scrub a value before the AI reads it
Personal contact data	Phone and email in profiles and addresses	Switching a masking rule on and off to see the difference
Card handling	Accepts a card number once, returns only the brand and last four digits	Never echoing secrets; a guardrail on card numbers typed into chat
Tokens in answers	A response that happens to contain a token	The built-in token pattern

8. Failures on purpose

One endpoint that fails on request is worth more than ten that usually work: for example /_test/fail?status=503&delay=3.

Status	When it should happen	What it tests in Frege
400	Broken JSON, an unknown field	The AI correcting its request
401 and 403	Not signed in; wrong role	Renewing a credential versus explaining a missing permission
404 and 409	Missing record; wrong state	Honest, specific explanations
422	Invalid fields, all listed together	The AI fixing every field at once
429	Too many attempts, with Retry-After	Waiting instead of hammering
500 and 503	A bug; a flaky service that works on the second try	Failure reasons in the audit log
Slow or unreachable	A long delay or a refused connection	Timeouts, classified reasons

9. Documentation and the spec itself

Two things are not endpoints but decide how good everything else feels.

A documentation site. Write real policy pages: refund rules with amounts, shipping thresholds, an error-code page, an FAQ in plain language. They give the Docs tab something to search and let evals ask questions whose answers contain numbers.

A careful OpenAPI file. The spec becomes the AI's tool list, so it should do the following.

- Give every operation a short summary and a description of when to use it.
- Use distinct, readable operation ids; snake_case reads best as tool names.
- Describe parameters with types, enums, examples and limits.
- Use JSON request bodies, since only JSON bodies become tool arguments.
- Mark sign-in operations as auth operations, and mark an operation that changes data but uses GET as mutating.
- Declare one https server address, because credentials are only sent to a pinned https address.
- Name the security scheme, and keep the file well under the 5 MB upload limit.

10. Helpers for the tester

Helper	Why
A webhook receiver that checks a signature and a timestamp	Receiving a Frege hook for real and proving the signature check
A demo inbox	Shows the one-time codes and the hook deliveries that would otherwise go to email
Seeded accounts, one per role	A tester can be a customer, staff and admin in a minute
A fixed one-time code mode	Scripted demos stay predictable

Helper	Why
Test cards that succeed, decline and run out of funds	Payment failures on demand
A way to reset the data	A demo can always start clean

11. The smallest set that covers everything

About 34 endpoints are enough to touch every feature above.

Group	Endpoints	Count
Authorization	register, login, request a code, verify a code, refresh, logout, authorize page, token endpoint, who am I	9
Roles	a public list, a staff list of everyone's records, an admin list of users	3
Reading	paged list, cursor list, filtered search, get one, HTML invoice, large export	6
Writing	create with a retry key, replace, update, delete, one action	5
Dangerous	pay, approve a refund, change a role	3
Slow work	start a job, check a job	2
Sensitive data	customer lookup with a taxpayer number, add a card	2
Failures	one endpoint that fails on request	1
Helpers	health check, webhook receiver, demo inbox	3
Total		**34**

Two rules keep the project honest. Make the behaviour real, not mocked: real signed tokens that really expire, real roles that really change what comes back. And keep every value made up, so a demo can never leak anything that matters.

When you choose the theme, pick one people care about, so the demo is remembered for the story and not only the features.